FPT UNIVERSITY – HO CHI MINH CITY CAMPUS

DEPARTMENT OF IC DESIGN AND DIGITAL AUTOMOTIVE

----- -----



FPT UNIVERSITY

Linux and open source platform

LAB 3 REPORT

Debugging kernel & phân tích ứng dụng hiệu năng nhúng

Name	Roll Number	Subject
Hoàng Ngọc Gia bão	SE203238	LOS201
Trần Hồ Khánh Tân	SE203740	LOS201
Nguyễn Trọng Hùng	SE204352	LOS201
Nguyễn Quế Vũ	SE204290	LOS201
Nguyễn Việt Anh	SE204354	LOS201
Hồ Phan Anh Tuấn	SE203437	LOS201

Lecturer : Phạm Thế Vinh

Section 1 hardware & software specification

- Kernel host 7.0.0-38-generic (Ubuntu, #38-Ubuntu SMP PREEMPT_DYNAMIC, build FriSep 4 09:10:14 UTC 2026, x86_64)

- GDB GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
- Valgrind valgrind-3.26.0
- perf perf version 7.0.14

A. GDB debug report

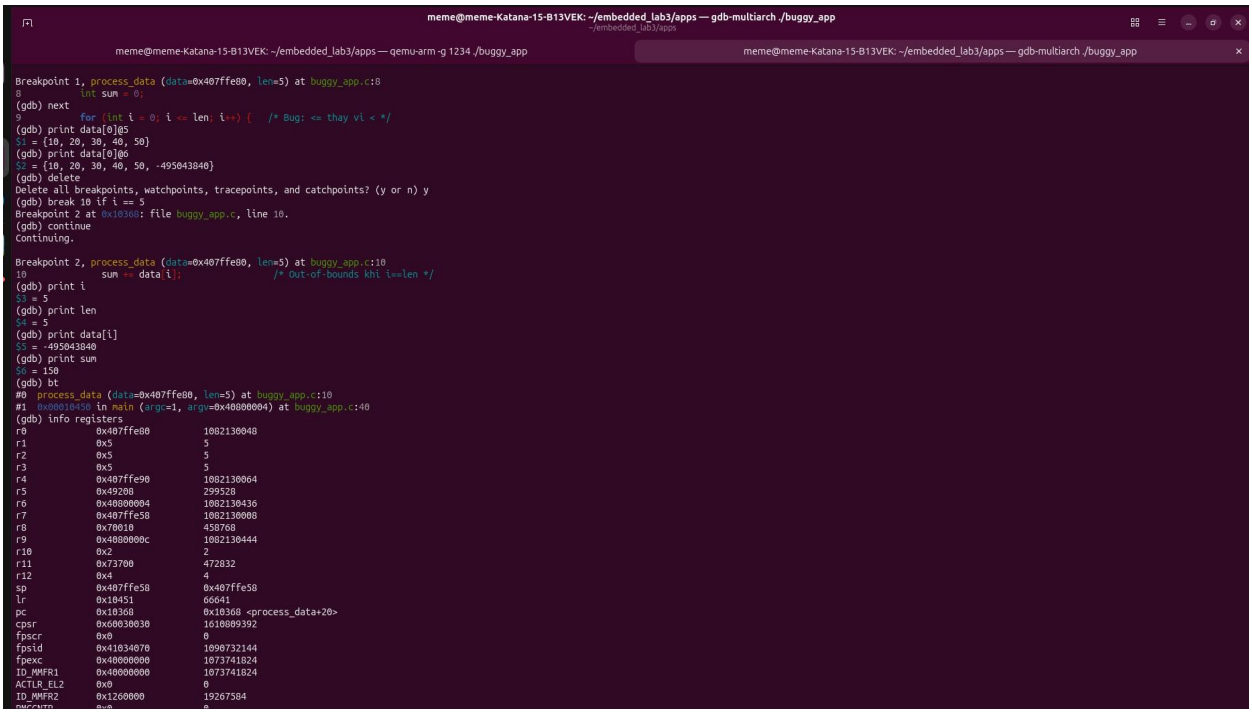
Bug 1: Out-of-bounds read (off-by-one) trong process_data

```
Quit anyway? (y or n) y
meme@meme-Katana-15-B13VEK:~/embedded_lab3/apps$ gdb-multiarch ./buggy_app
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./buggy_app...
(gdb) set debuginfod enabled off
(gdb) set architecture arm
The target architecture is set to "arm".
(gdb) set sysroot /usr/arm-linux-gnueabihf
(gdb) ❌ Quit
(gdb) set logging file gdb_session.log
(gdb) set logging on
Warning: 'set logging on', an alias for the command 'set logging enabled', is deprecated.
Use 'set logging enabled on'.

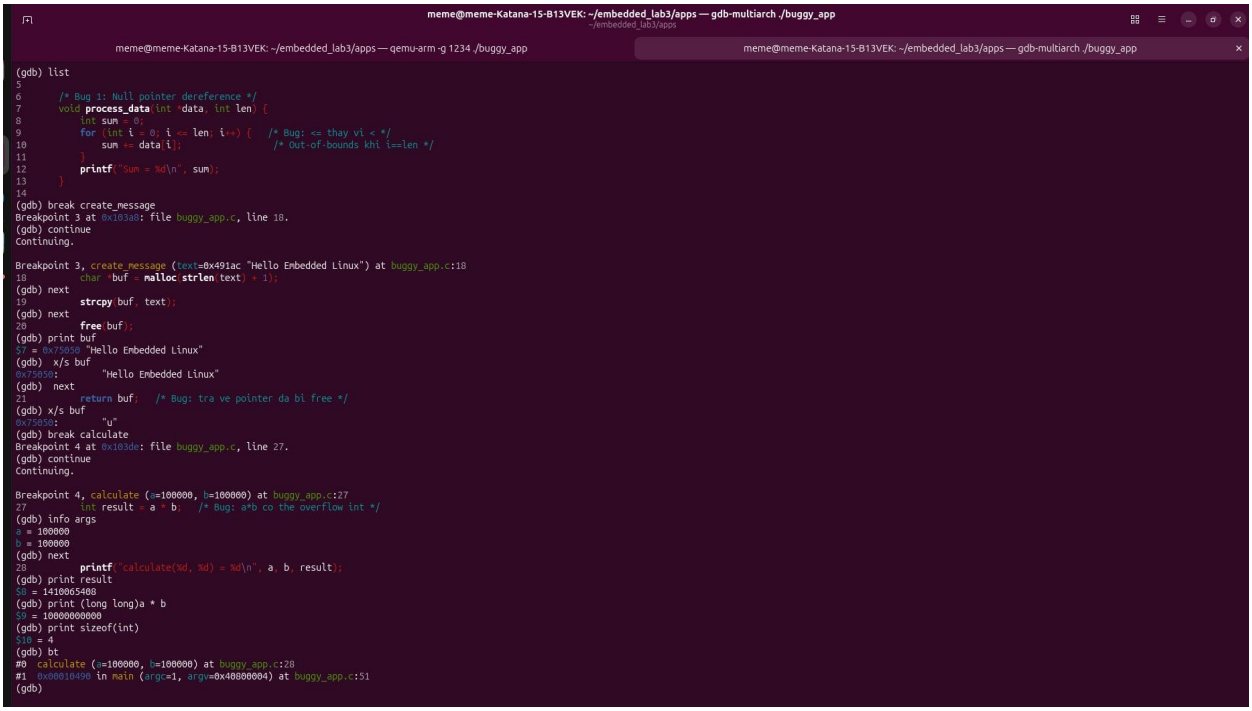
Copying output to gdb_session.log.
Copying debug output to gdb_session.log.
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
0x00010254 in _start ()
(gdb) break process_data
Breakpoint 1 at 0x1035e: file buggy_app.c, line 8.
(gdb) continue
Continuing.

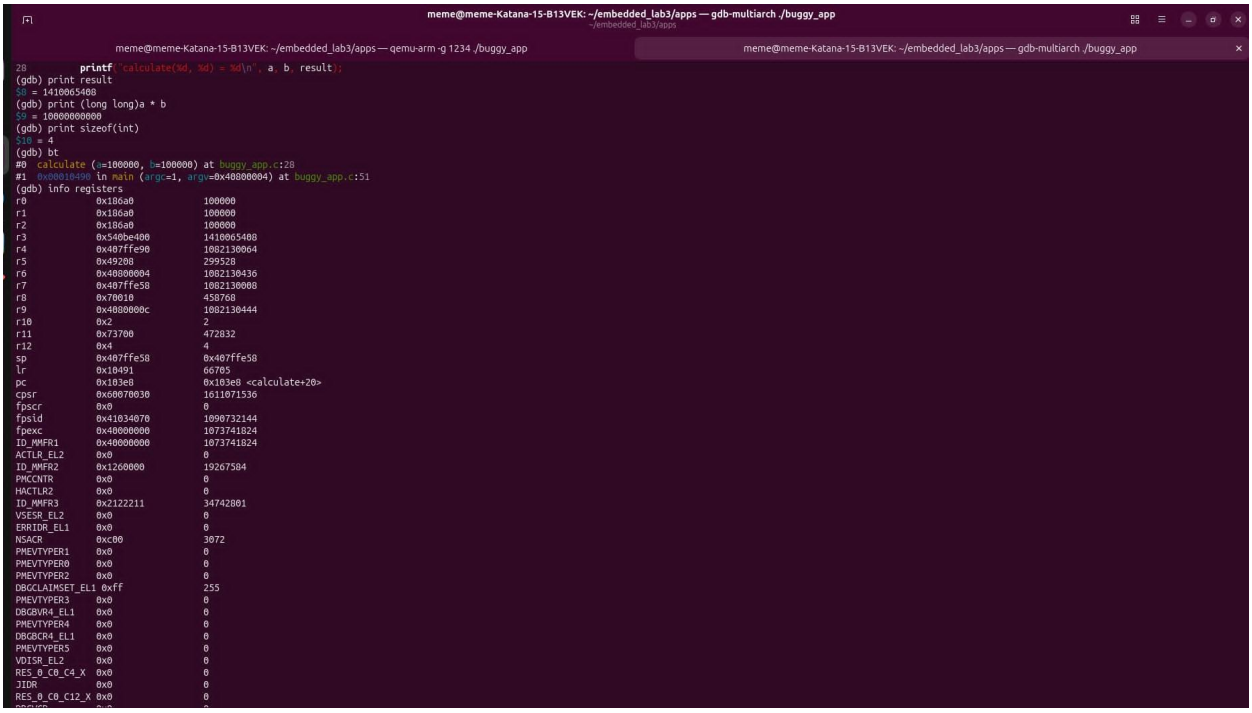
Breakpoint 1, process_data (data=0x407ffe80, len=5) at buggy_app.c:8
8           int sum = 0;
(gdb) next
9           for (int i = 0; i <= len; i++) {    /* Bug: <= thay vi < */
(gdb) print data[0]@5
$1 = {10, 20, 30, 40, 50}
(gdb) print data[0]@6
$2 = {10, 20, 30, 40, 50, -495043840}
(gdb)
```



Bug 2: Use-after-free trong create_message

Bug 3: Integer overflow trong calculate





Bug 1: Out-of-bounds read (off-by-one) trong process_data

Vị trí: buggy_app.c, vòng for ở dòng 9-10.

Nguyên nhân: Điều kiện lặp i <= len làm vòng lặp chạy thêm một lần với i == len. Mảng arr có 5 phần tử (chỉ số 0 đến 4) nhưng data[5] vẫn bị đọc, tức là đọc ra ngoài vùng nhớ của mảng. Giá trị đọc được là rác trên stack, làm sum sai và hành vi chương trình không xác định.

Cách sửa: Đổi điều kiện thành i < len.

```
for (int i = 0; i < len; i++) {
    sum += data[i];
}
```

Bug 2: Use-after-free trong create_message

Vị trí: buggy_app.c, dòng 20-21 (free(buf); return buf;).

Nguyên nhân: Hàm giải phóng buf rồi vẫn trả con trỏ đó về cho main. Sau free, vùng nhớ thuộc về bộ cấp phát và bị ghi đè dữ liệu quản lý (GDB cho thấy nội dung chỉ còn "!\004"). main dùng msg để in nên đọc dữ liệu đã bị hỏng (dangling pointer).

Cách sửa: Không free trong hàm. Hàm trả về vùng nhớ cấp phát động, còn nơi gọi chịu trách nhiệm giải phóng sau khi dùng xong.

```
char *create_message(const char *text) {
```

```
    char *buf = malloc(strlen(text) + 1);
    if (buf == NULL) return NULL;
    strcpy(buf, text);
    return buf;             /* không free ở đây */
}

/* trong main */
char *msg = create_message("Hello Embedded Linux");
printf("Message: %s\n", msg);
free(msg);                  /* free sau khi dùng xong */
msg = NULL;                 /* tránh dangling pointer */
```

Bug 3: Integer overflow trong calculate

Vị trí: buggy_app.c, dòng int result = a * b;.

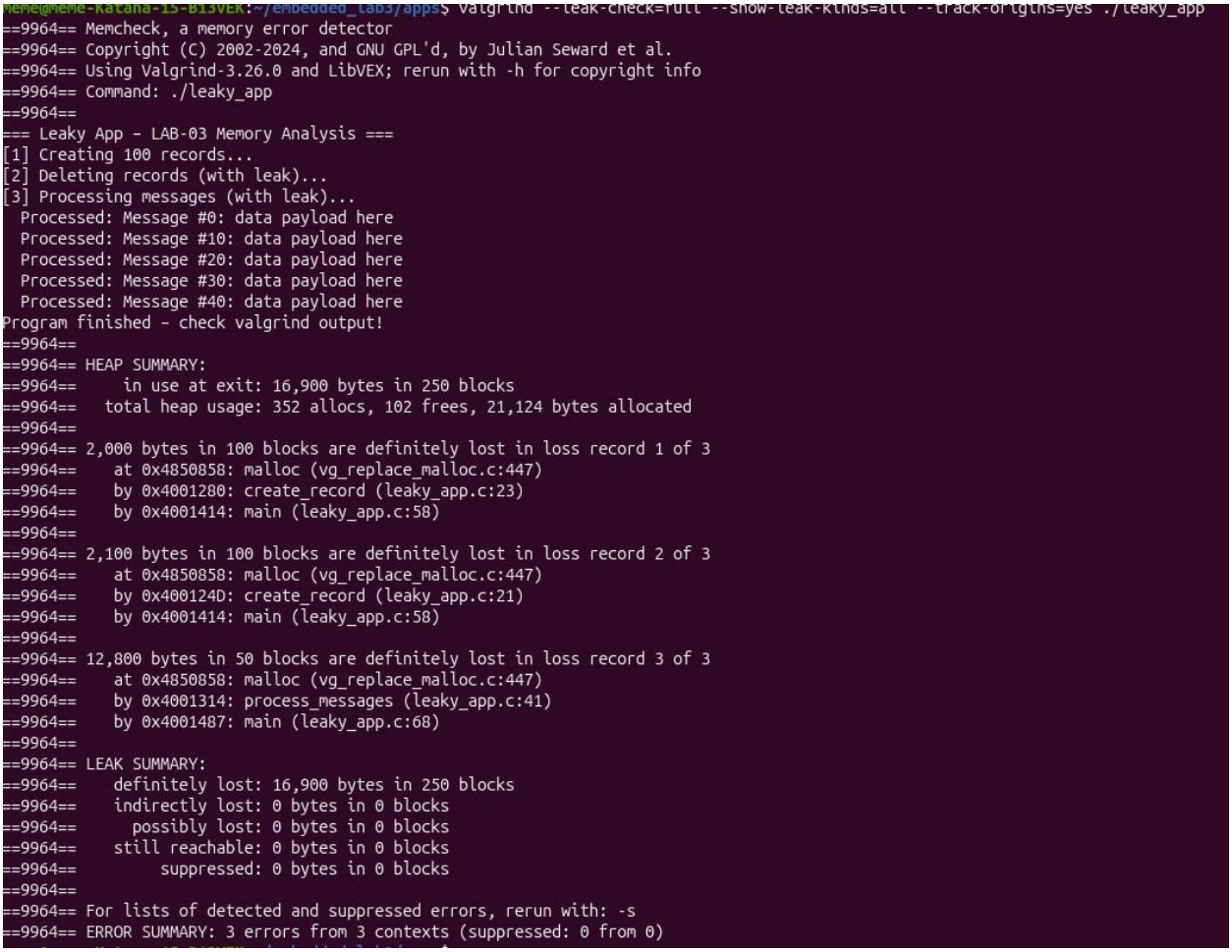
Nguyên nhân: calculate(100000, 100000) cho kết quả đúng là 10^10, vượt INT_MAX (2.147.483.647) của kiểu int 32 bit. Phép nhân bị tràn, kết quả bị cắt còn 1410065408. Trong C, tràn số nguyên có dấu là hành vi không xác định và không báo lỗi.

Cách sửa: Dùng kiểu 64 bit để tính và lưu kết quả.

```
#include <stdint.h>

int64_t calculate(int a, int b) {
    int64_t result = (int64_t)a * b;   /* ép kiểu trước khi nhân */
    printf("calculate(%d, %d) = %lld\n", a, b, (long long)result);
    return result;
}
```

B.Memory leak Anlysis report



Leak 1 và 2: r->name và r->scores.

- malloc được gọi trong create_record() (leaky_app.c:21 và :23), do main gọi ở dòng 58 trong vòng lặp tạo 100 record.
- Không được free vì delete_record() chỉ gọi free(r), còn free(r->name) và free(r->scores) bị comment out. free(r) chỉ giải phóng struct Record (khối đã được tính là freed), không giải phóng các con trỏ bên trong nó.
- Sau khi free(r), con trỏ name và scores không còn ai giữ, nên Valgrind xếp là definitely lost.

Leak 3: buf.

- malloc(MSG_SIZE) trong process_messages() (leaky_app.c:41), do main gọi ở dòng 68 với count = 50.
- Biến buf là biến cục bộ trong vòng lặp. Sau mỗi vòng, buf bị ghi đè bằng giá trị mới, và free(buf) bị thiếu, nên 50 buffer 256 bytes đều mất con trỏ.

```
==10191== Memcheck, a memory error detector
==10191== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==10191== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==10191== Command: ./leaky_app
==10191==
=== Leaky App - LAB-03 Memory Analysis ===
[1] Creating 100 records...
[2] Deleting records (with leak)...
[3] Processing messages (with leak)...
  Processed: Message #0: data payload here
  Processed: Message #10: data payload here
  Processed: Message #20: data payload here
  Processed: Message #30: data payload here
  Processed: Message #40: data payload here
Program finished - check valgrind output!
==10191==
==10191== HEAP SUMMARY:
==10191==     in use at exit: 0 bytes in 0 blocks
==10191==   total heap usage: 352 allocs, 352 frees, 21,124 bytes allocated
==10191==
==10191== All heap blocks were freed -- no leaks are possible
==10191==
==10191== For lists of detected and suppressed errors, rerun with: -s
==10191== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

C. **Profiling Report**

```
 % time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 49.30    0.001373        1373         1           execve
 25.13    0.000700          50        14           mmap
  3.91    0.000109          36         3           munmap
  3.70    0.000103          25         4           openat
  3.45    0.000096           1        75           write
  3.02    0.000084          16         5           fstat
  1.94    0.000054          13         4           close
  1.69    0.000047          11         4           mprotect
  1.62    0.000045          22         2           read
  1.51    0.000042          14         3           brk
  1.01    0.000028          14         2           pread64
  0.97    0.000027          27         1         1 access
  0.83    0.000023          23         1           arch_prctl
  0.65    0.000018          18         1           set_tid_address
  0.65    0.000018          18         1           set_robust_list
  0.65    0.000018          18         1           rseq
  0.00    0.000000           0         1           prlimit64
  0.00    0.000000           0         1           getrandom
------ ----------- ----------- --------- --------- ----------------
100.00    0.002785          22       124         1 total
meme@meme-Katana-15-B13VEK:~/embedded_lab3/apps$
```

Kết quả strace: execve và mmap chiếm hơn 74% thời gian syscall, nhưng đây là chi phí khởi động một lần (nạp chương trình, ánh xạ thư viện động). Toàn bộ syscall chỉ tốn khoảng 2.8 ms, không có lời gọi đọc/ghi nào chậm bất thường. Hàm write được gọi 75 lần nhưng mỗi lần chỉ khoảng 1 µs.

```
meme@meme-Katana-15-B13VEK:~/embedded_lab3/logs$ sudo perf report --stdio --no-children -g none -i perf.data | head -30
[sudo: authenticate] Password:
# To display the perf.data header info, please use --header/--header-only options.
#
#
# Total Lost Samples: 0
#
# Samples: 48  of event 'cpu_atom/cycles/P'
# Event count (approx.): 24996523
#
# Overhead  Command      Shared Object      Symbol
# ........  ...........  .................  ................................
#
    43.07%  perf_target  libm.so.6         [.] __ieee754_log_fma
    33.92%  perf_target  perf_target       [.] heavy_compute
     4.91%  perf_target  perf_target       [.] log@plt
     2.56%  perf_target  [kernel.kallsyms] [k] drm_send_event_helper
     2.43%  perf_target  [kernel.kallsyms] [k] ktime_get
     2.20%  perf_target  [i915]            [k] 0x0000000000003dbf
     1.67%  perf_target  [kernel.kallsyms] [k] kernel_init_pages
     1.57%  perf_target  perf_target       [.] main
     1.54%  perf_target  [kernel.kallsyms] [k] get_mem_cgroup_from_mm
     1.46%  perf_target  [kernel.kallsyms] [k] percpu_counter_add_batch
     1.34%  perf_target  [kernel.kallsyms] [k] __handle_mm_fault
     1.20%  perf_target  [kernel.kallsyms] [k] tlb_gather_mmu
     0.92%  perf_target  [kernel.kallsyms] [k] entry_SYSRETQ_unsafe_stack
     0.72%  perf_target  [kernel.kallsyms] [k] arch_get_unmapped_area_topdown
     0.50%  perf_target  [kernel.kallsyms] [k] _find_first_zero_bit


# Samples: 72  of event 'cpu_core/cycles/P'
# Event count (approx.): 60623940
```

Kết quả perf: Thời gian CPU tập trung vào đúng chuỗi tính toán của chương trình: heavy_compute (33.92%) gọi log qua PLT (4.91%) vào __ieee754_log_fma trong libm.so.6 (43.07%). Cộng lại, hơn 80% CPU nằm trong heavy_compute và hàm log mà nó gọi. Các dòng drm_send_event_helper, ktime_get, i915 là ngắt đồ họa và hàm kernel xảy ra ngẫu nhiên lúc đo, không thuộc chương trình nên bỏ qua khi phân tích.

Kết luận bottleneck: perf_target bị nghẽn CPU (CPU-bound), không phải I/O**.** Lý do:

- strace cho thấy thời gian ở kernel/syscall rất nhỏ (~2.8 ms), không có I/O chậm.
- perf cho thấy phần lớn chu kỳ CPU nằm ở user space, trong hàm tính toán heavy_compute và log() của libm.

Đề xuất tối ưu

1. **Giảm số lần gọi** log(): đưa các phép log bất biến ra ngoài vòng lặp, lưu cache hoặc tính trước các giá trị lặp lại.
2. **Thay thế hàm** log: dùng logf (float) nếu đủ độ chính xác, hoặc bảng tra (lookup table) / xấp xỉ đa thức cho phép tính này.
3. **Tối ưu biên dịch**: dùng -O2/-O3, -march=native để vector hóa (SIMD), và -ffast-math nếu chấp nhận sai số nhỏ.
4. **Tối ưu thuật toán trong** heavy_compute: giảm độ phức tạp vòng lặp, hoặc chia việc cho nhiều luồng (đa luồng) nếu các phép tính độc lập.
5. **Gom** write: dùng buffer lớn hơn (setvbuf/fwrite) để giảm 75 lời gọi. Lợi ích nhỏ vì syscall không phải bottleneck.

6. **Đo lại để kiểm chứng**: chạy lại perf record sau khi tối ưu, tăng tần số lấy mẫu (-F 999) hoặc chạy lâu hơn, vì lần đo hiện tại chỉ có 48 samples (core atom) và 72 samples (core P), còn ít.

D. Real-Time Latency Report

1. Bảng thống kê latency trên Standard Kernel

Đo bằng cyclictest (4 thread, SCHED_FIFO priority 80, interval 1000 µs, 10 000 vòng lặp/thread, --mlockall). Đơn vị: µs.

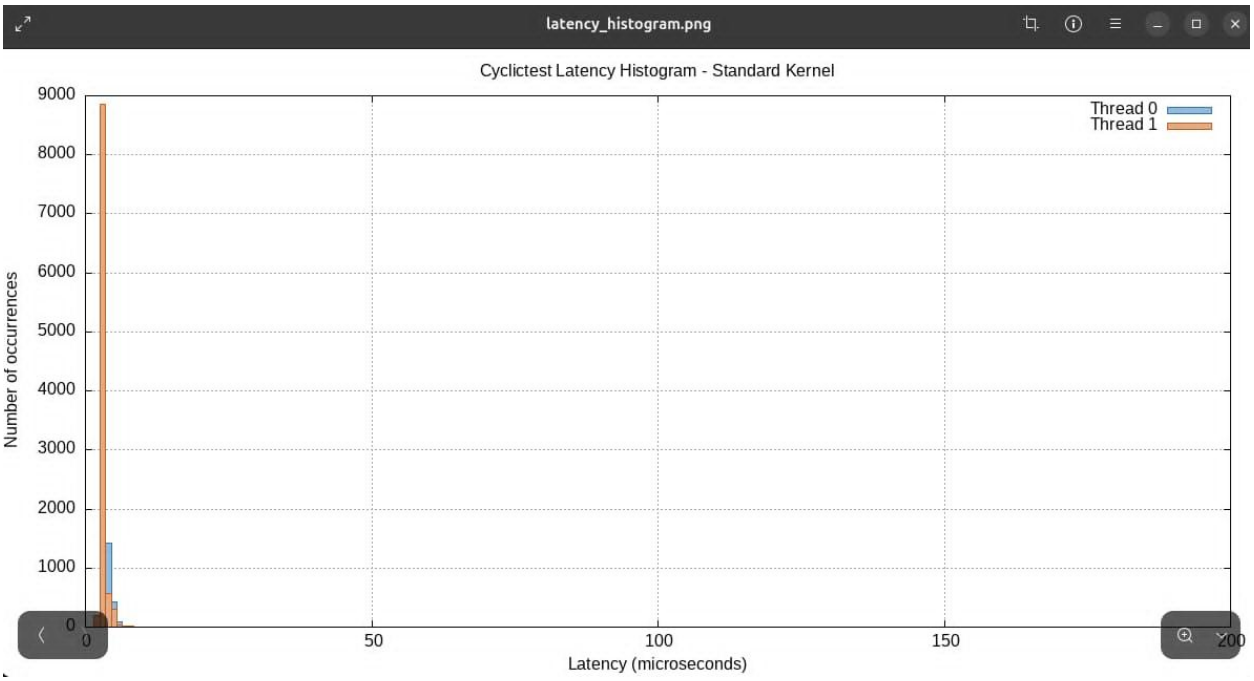
Condition no load

Thread	Min	Avg	Max
T0	2	3	122
T1	2	3	101
T2	2	3	122
T3	2	3	104
Tổng hợp	2	3	122

Condition load

Thread	Min	Avg	Max
T0	2	3	134
T1	2	3	1322
T2	2	3	1179
T3	2	3	140
Tổng hợp	2	3	1322

2. Histogram (gnuplot)
3.



3. Vì sao Max latency quan trọng hơn Avg trong hệ real-time

Hệ real-time được đánh giá bằng việc **mọi** tác vụ có kịp deadline hay không, chứ không phải bằng hiệu năng trung bình. Avg bị "pha loãng" bởi hàng nghìn mẫu nhanh nên che mất các lần trễ hiếm nhưng lớn. Max cho biết trường hợp xấu nhất, là giá trị quyết định deadline có bị lỡ hay không.

Số liệu của nhóm minh họa rõ điều này: Avg chỉ 3 µs ở cả hai điều kiện, nhưng Max lên tới 1322 µs khi có tải, gấp hơn 400 lần Avg. Với vòng điều khiển cần phản hồi trong 500 µs, chỉ một lần trễ 1322 µs là đủ gây lỡ deadline, làm robot giật hoặc mất ổn định, dù trung bình vẫn rất tốt. Vì Max là sự kiện hiếm nên nó cần đo đủ lâu mới bắt được, và nó biến động giữa các lần đo.

4. Cơ chế PREEMPT_RT giúp giảm Max latency

Trên standard kernel, tác vụ ưu tiên cao có thể phải chờ lâu vì kernel có nhiều đoạn không thể bị ngắt: vùng giữ spinlock, trình xử lý ngắt chạy khi ngắt bị tắt, và softirq. PREEMPT_RT xử lý các điểm này bằng:

- **Kernel gần như hoàn toàn preemptible:** hầu hết vùng găng có thể bị tác vụ ưu tiên cao chen ngang.
- **Spinlock chuyển thành rt_mutex:** tác vụ chờ khóa có thể ngủ và bị preempt thay vì chiếm CPU bận chờ.
- **Priority inheritance:** tác vụ giữ khóa được nâng ưu tiên tạm thời, tránh priority inversion.
- **Threaded interrupt:** trình xử lý ngắt chạy thành kernel thread có priority, lập lịch được như tác vụ thường nên không chặn tác vụ real-time.
- **High-resolution timer:** thời điểm đánh thức chính xác hơn, giảm jitter.

Kết quả là thời gian chờ tối đa bị chặn trên và dự đoán được, nên Max latency giảm mạnh dù Avg có thể chỉ thay đổi ít. Cái giá là throughput tổng thể thấp hơn đôi chút.

5. Đề xuất kernel cho hệ thống điều khiển robot (< 500 µs)

Nên dùng PREEMPT_RT.

- Standard kernel không cam kết giới hạn worst-case. Trong phép đo của nhóm, Max có tải đạt 1322 µs, vượt gấp hơn 2,5 lần yêu cầu 500 µs.
- PREEMPT_RT giới hạn được worst-case latency nên đáp ứng yêu cầu một cách đáng tin cậy. *(Nếu có số đo RT, điền thêm: "Trên PREEMPT_RT, Max đo được là … µs, thấp hơn 500 µs.")*
- Khi triển khai nên kết hợp: chạy tác vụ điều khiển với SCHED_FIFO, dùng mlockall để tránh page fault, và nếu cần thì cô lập CPU riêng cho vòng điều khiển.

Với robot, tính tất định (deterministic) quan trọng hơn throughput, nên đánh đổi này là xứng đáng.